SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	Arvind.S	**Reg. No.:**	192525192
Section / Batch:		**Date:**	26/08/2026

Code of Conduct

I Arvind.S (192525192) *(Name / Reg No)*

Certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Arvind.S

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative Implementations	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature:	Signature:
Date:	Date:	Date:

C. Mokshitha Reddy
192571054

Linear Probing vs Separate chaining

Point	Linear probing	Separate chaining
Clustering	Suffers from Primary clustering	Less clustering
Memory usage	uses less extra memory	Needs extra memory for linked lists
High load factor	Performance decreases quickly	Handles high load better
Large-scale use	Less suitable when table becomes full	More suitable due to flexible storage.

Separate chaining is generally better for large-scale applications because it handles collisions and high load factors efficiently.

2. Quick Sort vs Merge Sort for 10 Million Linked-List Nodes.

Point	Quick Sort	Merge Sort
Time	Average: O(n log n), Worst: O(n²)	Always O(n log n)
Extra space	Recursion stack required	(o log n) stack + merging.
Linked Lists	Difficult due to Partitioning	Very efficient Using pointer manipulation
Stability	Usually not stable	stable

→ Merge sort is preferred for linked lists because it does not require random access and can efficently split and merge nodes.

For arrays:

Quick sort is often preferred in practice because of good cache performance and low extra memory.

Quadratic probing vs Double Hashing

Point	Quadratic probing	Double Hashing
Clustering	Reduces primary clustering but may have secondary clustering.	Minimizes both types of clustering
Collision efficiency	Good	Better
Dense hash table	Less suitable at very high load	More suitable
Probe Sequence	uses quadratic increments	uses a second hash function

4. Best - case vs Worst - case complexity

Algorithm	Best Case	Worst case
Bubble sort	$O(n)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n^2)$

a) Practical significance:-

Best case occurs when the input is already favorable; worst case occurs with unfavourable input.

b) Impact on selection:-

For large datasets, algorithms with O(n log n) worst case complexity, such as Merge sort, are generally safer and more predictable.

5. Recursive Vs Iterative Quick sort

Point	Recursive Quick sort	Iterative Quick sort
Space	Uses function Call stack	Uses explicit stack
Implement-ation	Easier and simpler	More complex
Stack overflow	Possible for bad Partitions	Can be controlled using an explicit stack.
Performance	Good average Performance	Similar average performance.

→ Recursive Quick sort is easier to implement, but Iterative Quick sort provides better control over stack usage and reduses the risk of Stack overflow.